

```
1 #ifndef GROUP_H
2 #define GROUP_H
3
4 #include "linked_list.h"
5 #include "gen_table.h"
6 #include <assert.h>
7 #include <stdbool.h>
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <time.h>
11 #include <math.h>
12
13 typedef struct group {
14     char *name; // Nom
15     int n; // Cardinal du groupe
16     int **table; /* Table de Cayley:
17                  le groupe est assimilé
18                  à [0, 1, ..., n-1]
19                  */
20 } group_t;
21
22 typedef struct group_data {
23     gentabl_t* rel; // Table des décompositions
24     list_t* gen; // Liste des générateurs
25 } gdata_t;
26
27 typedef struct decomp_abelian_s {
28     int n;
29     int num_basis;
30     int* basis;
31     int** basis_decomps;
32 } decomp_t;
33
34 void add_sum_to_decomp(int x, int y, decomp_t* d);
35
36 group_t *group_empty(int n);
37 void group_name(group_t *g, char *s);
38 void group_free(group_t *g);
39 void group_print(group_t *g);
40
41 gdata_t* create_empty_gdata();
42 gdata_t* parse_entries(list_t* gen, gentabl_t* rel);
43 void free_gdata(gdata_t* gdata);
44
45 list_t* get_generator(gdata_t* gdata);
46 gentabl_t* get_rel(gdata_t* gdata);
47
48 group_t *from_matrix(int **matrix);
49 group_t *copy_group(group_t *g);
50
51 int calculate(group_t* g, int x, int y);
52 bool is_neutral(group_t* g, int x);
53 int get_neutral(group_t* g);
54 int calculate_order(group_t* g, int x);
55 int* get_orders_list(group_t* g);
56
57 int calc_generated_subgroup(group_t* g, int gen, list_t* generators, gentabl_t*
decomps);
58 gdata_t* calc_gen_and_rel(group_t* g);
59 list_t* get_cyclic_subgroup_element(group_t* g, int x, int e);
```

```
60 bool are_isomorphic(group_t* g1, group_t* g2);
61 bool are_isomorphic_with_order(group_t* g1, group_t* g2);
62
63 void print_array(int* arr, int n);
64
65 bool are_isomorphic_with_order_custom_gen(group_t* g1, group_t* g2, int* generators,
    int num_gen);
66
67 bool are_isomorphic_probabilistic(group_t* g1, group_t* g2, int max_attempts);
68 bool are_isomorphic_hybrid(group_t* g1, group_t* g2, long long
    deterministic_threshold, int probabilistic_attempts);
69 bool are_isomorphic_adaptive(group_t* g1, group_t* g2);
70 #endif
```